VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Microprocessors – Microcontrollers

Laboratory Report 3

Buttons/Switches

Advisor(s): Phan Văn Sỹ

Student(s): Vũ Thiện Nhân – 2452888

HO CHI MINH CITY, NOVEMBER 2026

Table of Contents

1. Introduction 3
2. Hardware Design 3
   2.1 Schematic for the Traffic Light at a Crossing 3
   2.2 Pin Assignment 4
3. Exercise 1: Sketch an FSM 5
4. Exercise 2: Proteus Schematic 7
5. Exercise 3: Create STM32 Project 8
6. Exercise 4: Modify Timer Parameters 9
7. Exercise 5: Adding Code for Button Debouncing 11
8. Exercise 6: Adding Code for Displaying Modes 14
9. Exercise 7: Increasing the RED Duration 18
10. Exercise 8: Increasing the YELLOW (Amber) Duration 19
11. Exercise 9: Increasing the GREEN Duration 20
12. Exercise 10: Finishing the Project 20

1. Introduction

This report presents the implementation of Laboratory 3 (Buttons/Switches) of the Microprocessors – Microcontrollers course, using the STM32F103C6 microcontroller simulated in Proteus 8 Professional and programmed with STM32CubeIDE (HAL library).

The laboratory builds the controller of a traffic light at a crossing. The system consists of 12 traffic LEDs (RED / YELLOW / GREEN for each of the North, South, East and West directions), four 7-segment LEDs (two digits for each road) and three push buttons. The application has four modes, selected with the first button:

- **Mode 1 – normal mode:** the traffic lights run and the 7-segment LEDs show the remaining time of each road.
- **Mode 2 – modify the RED duration:** all red LEDs blink at 2 Hz; the second button increases the value (1–99) and the third button sets it.
- **Mode 3 – modify the YELLOW (amber) duration:** same behaviour with the yellow LEDs.
- **Mode 4 – modify the GREEN duration:** same behaviour with the green LEDs.

Road A (North–South) is the main road, the one whose durations are edited. The durations of road B (East–West) are derived from road A so that the two roads can never be green at the same time: RED(B) = GREEN(A) + YELLOW(A), GREEN(B) = RED(A) − YELLOW(A) and YELLOW(B) = YELLOW(A).

The software follows the techniques of the lab handout: a timer interrupt every 10 ms, button reading with a two-sample debouncing filter, a finite state machine (FSM) per button for short/long presses, and software timers for every time-dependent behaviour. As in Laboratory 1, all LEDs are active-low: the anode is tied to 3.3 V and the cathode to a GPIO pin, so driving the pin to logic 0 (GPIO_PIN_RESET) turns the LED on. The same principle applies to the common-anode 7-segment LEDs and, because the buttons are wired to ground with the internal pull-up resistor enabled, a pressed button reads logic 0.

2. Hardware Design

Figure 2.1 shows the Proteus schematic used for all the exercises of this laboratory, and Tables 2.1 and 2.2 list the pin assignment.

2.1 Schematic for the Traffic Light at a Crossing

The schematic contains the STM32F103C6 (U1), twelve LEDs D1–D12, four common-anode 7-segment LEDs driven through the PNP transistors Q1–Q4, and three push buttons. Every signal is connected to the microcontroller through a net label that has the same name as the CubeMX user label of the corresponding pin.

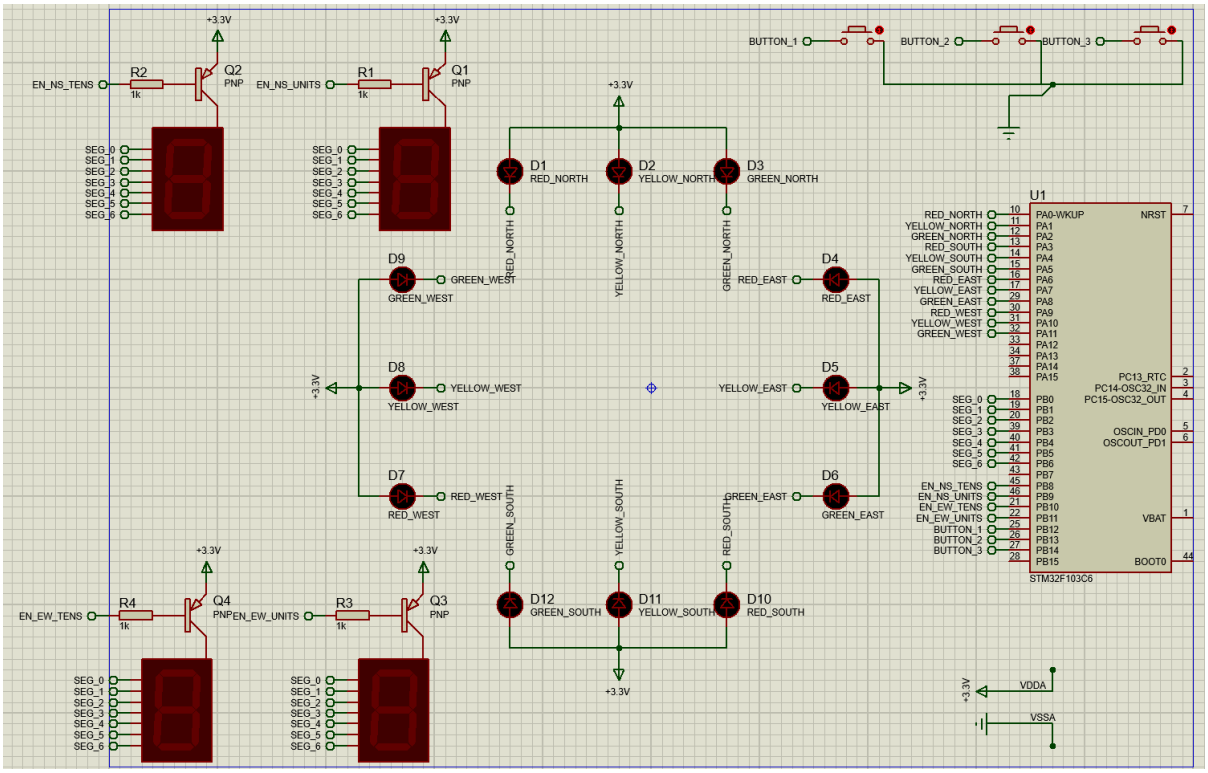


Figure 2.1: Proteus schematic for the traffic light at a crossing (12 LEDs, 4 seven-segment LEDs, 3 buttons)

2.2 Pin Assignment

Table 2.1 — Pin assignment of the 12 traffic LEDs (GPIO_Output, active-low)

Direction	RED	YELLOW	GREEN
North	PA0 (RED_NORTH)	PA1 (YELLOW_NORTH)	PA2 (GREEN_NORTH)
South	PA3 (RED_SOUTH)	PA4 (YELLOW_SOUTH)	PA5 (GREEN_SOUTH)
East	PA6 (RED_EAST)	PA7 (YELLOW_EAST)	PA8 (GREEN_EAST)
West	PA9 (RED_WEST)	PA10 (YELLOW_WEST)	PA11 (GREEN_WEST)

Table 2.2 — Pin assignment of the 7-segment LEDs and the buttons

Function	Label	Pin	Configuration
Segments a–g	SEG_0 … SEG_6	PB0 … PB6	GPIO_Output, lit at 0
Road A tens / units digit enable	EN_NS_TENS / EN_NS_UNITS	PB8 / PB9	GPIO_Output, digit on at 0 (PNP)
Road B tens / units digit enable	EN_EW_TENS / EN_EW_UNITS	PB10 / PB11	GPIO_Output, digit on at 0 (PNP)
Button 1 – select mode	BUTTON_1	PB12	GPIO_Input, pull-up, pressed = 0
Button 2 – increase value	BUTTON_2	PB13	GPIO_Input, pull-up, pressed = 0
Button 3 – set value	BUTTON_3	PB14	GPIO_Input, pull-up, pressed = 0

3. Exercise 1: Sketch an FSM

The solution follows the Input/Output Processing Pattern of the handout (Figure 1.5 of the lab): an input driver stores the debounced state of the buttons in an input buffer, a processing module (the FSMs) turns the buffer into events and updates the output buffer, and an output driver transfers the output buffer to the LEDs. Figure 3.1 shows the resulting software architecture.

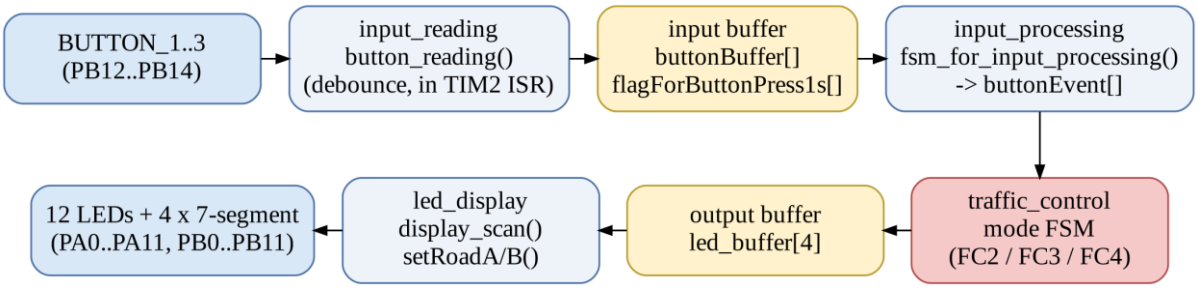


Figure 3.1: Software architecture of the application

The program is split into four routines, named FC1 to FC4 in the source code (Table 3.1).

Table 3.1 — Routines of the application

Routine	Function	File	Role
FC1	HAL_TIM_PeriodElapsedCallback()	timer.c	Every 10 ms: read and debounce the 3 buttons, tick the software timers
FC2	traffic_mode_control()	traffic_control.c	BUTTON_1 selects the next mode (1 → 2 → 3 → 4 → 1)
FC3	traffic_normal()	traffic_control.c	Mode 1: run the traffic lights
FC4	traffic_edit()	traffic_control.c	Modes 2, 3, 4: blink, change and set one duration

Mode FSM. The application has four states, one per mode (Figure 3.2). BUTTON_1 moves from mode 1 to 2, 3, 4 and back to 1. In modes 2–4 BUTTON_2 increases the draft value (1–99, 99 wraps to 1) and BUTTON_3 validates the draft, stores it and returns to mode 1. Leaving a mode with BUTTON_1 discards the draft value.

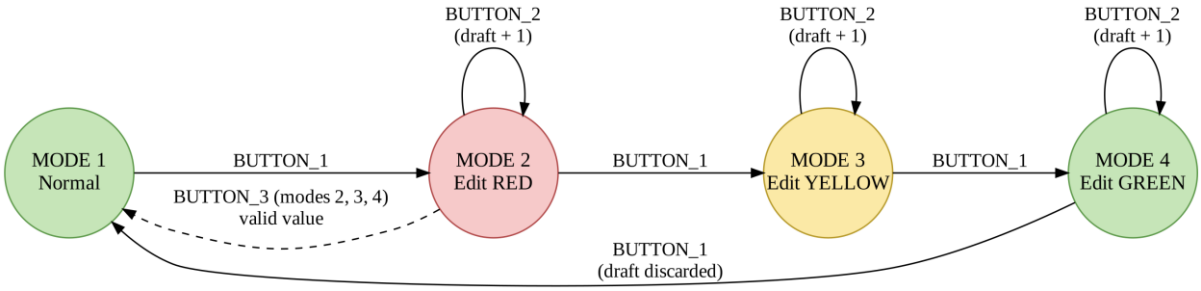


Figure 3.2: FSM of the four modes

Button FSM. Every button has the same three-state FSM as Figure 1.9 of the handout (Figure 3.3). State 1 fires exactly one event when the button is pressed; state 2 is entered after one second and, for BUTTON_2 only, fires an event every 500 ms so that the value can be increased by holding the button.

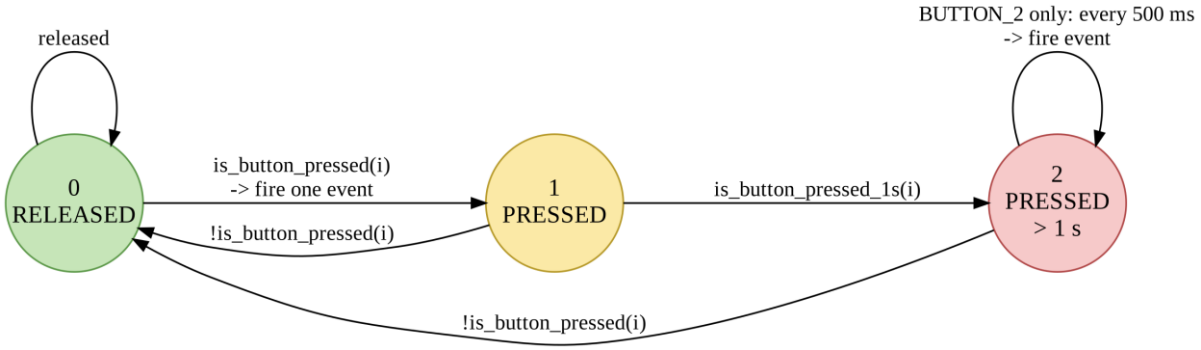


Figure 3.3: FSM used to process each button

Traffic light FSM (mode 1). The state of road A is one of three phases and a counter `remaining` holds the seconds left in the phase (Figure 3.4). With the durations R, Y and G of road A, road B is derived as follows:

- road A RED (R seconds): road B is GREEN for the first R − Y seconds, then YELLOW for the last Y seconds;
- road A GREEN (G seconds): road B is RED, its counter shows remaining + Y;
- road A YELLOW (Y seconds): road B is RED, its counter shows remaining.

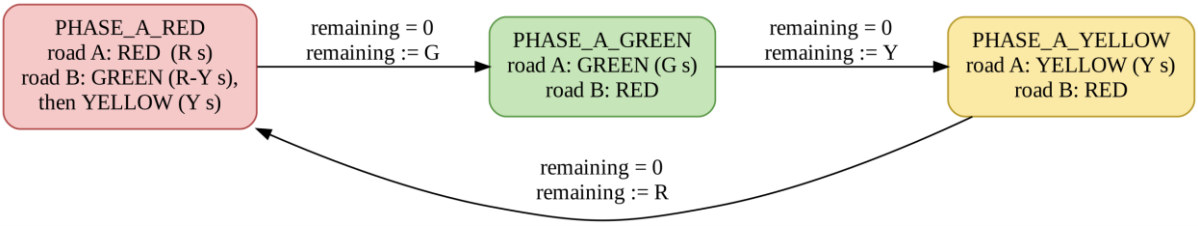


Figure 3.4: FSM of the traffic lights in mode 1

Both roads therefore have the same cycle of R + G + Y seconds. A new set of durations is accepted only if R > Y (so that the green time R − Y of road B is positive) and G + Y ≤ 99 (so that the red time of road B fits on two digits); otherwise BUTTON_3 is ignored and the system stays in the edit mode. Table 3.2 gives the timeline of the default durations R = 5, Y = 2, G = 3.

Table 3.2 — Timeline of one cycle with the default durations (R = 5, Y = 2, G = 3)

Time (s)	Road A light	Road A display	Road B light	Road B display
0	RED	05	GREEN	03
1	RED	04	GREEN	02
2	RED	03	GREEN	01
3	RED	02	YELLOW	02
4	RED	01	YELLOW	01
5	GREEN	03	RED	05
6	GREEN	02	RED	04
7	GREEN	01	RED	03
8	YELLOW	02	RED	02
9	YELLOW	01	RED	01

4. Exercise 2: Proteus Schematic

The schematic of Figure 2.1 was drawn in Proteus 8 Professional with the following design decisions:

- **Traffic LEDs.** The 12 LEDs D1–D12 have their anode on +3.3 V and their cathode on one pin of PA0–PA11, so each LED is lit when its pin is driven to 0. D1–D3 are the North LEDs, D4–D6 the East LEDs, D7–D9 the West LEDs and D10–D12 the South LEDs (red, yellow, green in each group).
- **7-segment LEDs.** The four displays are common-anode and share the seven segment lines SEG_0 … SEG_6 (PB0–PB6). Only one digit is lit at a time (time multiplexing), which needs 7 + 4 = 11 pins instead of 28.
- **Digit enable.** The common anode of each display is switched by a PNP transistor (Q1–Q4) whose emitter is on +3.3 V and whose base is driven by an enable pin through a 1 kΩ resistor (R1–R4); a digit is on when its enable pin is 0. Q2 and Q1 are

the tens and units digits of road A (EN_NS_TENS, EN_NS_UNITS); Q4 and Q3 are the tens and units digits of road B (EN_EW_TENS, EN_EW_UNITS).

- **Buttons.** BUTTON_1, BUTTON_2 and BUTTON_3 connect PB12, PB13 and PB14 to ground when pressed. No external resistor is used: the internal pull-up of the pins is enabled in the firmware, so a released button reads 1 and a pressed button reads 0.
- **Unused pins.** PA13 and PA14 are left free for the ST-Link (Serial Wire debug).

5. Exercise 3: Create STM32 Project

A new project was created in STM32CubeIDE for the STM32F103C6 with the configuration of Table 5.1. Every pin received a user label identical to the net label of the schematic so that CubeMX generates the macros `<LABEL>_Pin` and `<LABEL>_GPIO_Port` in `main.h`.

Table 5.1 — CubeMX configuration of the project

Item	Setting
Clock	HSI 8 MHz, PLL disabled (SYSCLK = HCLK = PCLK1 = PCLK2 = 8 MHz)
Debug	Serial Wire (PA13, PA14 reserved for the ST-Link)
TIM2	Clock source: internal clock; Counter mode: up; Prescaler = 7999; Counter Period = 9
Timer interrupt period	8 MHz / (7999 + 1) / (9 + 1) = 100 Hz → 10 ms; TIM2 global interrupt enabled in NVIC
PA0 … PA11, PB0 … PB6, PB8 … PB11	GPIO_Output, push-pull, no pull, low speed (12 LEDs, 7 segments, 4 digit enables)
PB12, PB13, PB14	GPIO_Input, internal pull-up (BUTTON_1, BUTTON_2, BUTTON_3)

Listing 5.1 — Exercise 3: TIM2 configuration generated by CubeMX (main.c)

```
htim2.Instance = TIM2;
htim2.Init.Prescaler = 7999;
htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
htim2.Init.Period = 9;
htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
if (HAL_TIM_Base_Init(&htim2) != HAL_OK)
{
  Error_Handler();
}
```

Listing 5.2 — Exercise 3: GPIO configuration generated by CubeMX (main.c)

```
  GPIO_InitStruct.Pin = RED_NORTH_Pin|YELLOW_NORTH_Pin|GREEN_NORTH_Pin|RED_SOUTH_Pin
                        |YELLOW_SOUTH_Pin|GREEN_SOUTH_Pin|RED_EAST_Pin|YELLOW_EAST_Pin
                        |GREEN_EAST_Pin|RED_WEST_Pin|YELLOW_WEST_Pin|GREEN_WEST_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  GPIO_InitStruct.Pin = BUTTON_1_Pin|BUTTON_2_Pin|BUTTON_3_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_PULLUP;
  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
```

The user code is organised in separate source and header files, as in Figure 1.6 of the handout (Table 5.2). The timer callback `HAL_TIM_PeriodElapsedCallback()` is not written in `main.c` but in `timer.c`.

Table 5.2 — File organisation (Core/Src and Core/Inc)

File	Content
global.h	Constants shared by the whole project: timer cycle, time constants, modes, default durations, button indices
timer.c / timer.h	Software timers and the TIM2 interrupt callback (FC1)
input_reading.c / .h	Input driver: button reading, debouncing, 1-second detection
input_processing.c / .h	FSM of each button and the button events
led_display.c / .h	Output driver: traffic LEDs, 7-segment LEDs, output buffer, scanning
traffic_control.c / .h	Application: mode FSM, traffic lights, time editing (FC2, FC3, FC4)
main.c	Initialisation of the peripherals and the super loop

6. Exercise 4: Modify Timer Parameters

The aim of this exercise is that a change of the timer interrupt duration only needs a minimum of modifications and does not change the behaviour of the system. For this purpose the period of the interrupt is defined in only one place, the constant `TIMER_CYCLE` of `global.h`, which must be equal to the period configured in CubeMX. Every other duration of the program (one second, 250 ms blinking half-period, 500 ms auto-repeat, 10 ms digit scan, 1 s button hold) is written in milliseconds and converted into a number of timer ticks by `ms_to_ticks()`; no file contains a hard-coded tick count.

Listing 6.1 — Exercise 4: TIMER_CYCLE, ms_to_ticks() and time constants (global.h)

```
#ifndef TIMER_CYCLE
#define TIMER_CYCLE           10
#endif

/* Convert a duration in ms into a number of timer ticks (rounded, at least 1). */
static inline uint32_t ms_to_ticks(uint32_t ms)
{
    uint32_t ticks = (ms + TIMER_CYCLE / 2) / TIMER_CYCLE;
    return (ticks > 0) ? ticks : 1;
}
```

```
/* ---- time constants (ms) ------------------------------------------------- */
#define ONE_SECOND_MS          1000   /* traffic light tick in mode 1                  */
#define BLINK_HALF_PERIOD_MS   250    /* 2 Hz blinking = toggle every 250 ms           */
#define HOLD_TIME_MS           1000   /* hold a button longer than this -> auto repeat */
#define AUTO_INCREASE_MS       500    /* auto repeat period of button 2                */
#define SCAN_PERIOD_MS         10      /* time one 7-segment digit stays on             */
```

The software timers are defined in `timer.c`. `setTimer()` receives a duration in milliseconds, `timer_run()` is called once per interrupt, and `isTimerExpired()` returns 1 only once after the timer expires. The interrupts are masked for a moment in `setTimer()` and `isTimerExpired()` because the main loop and the interrupt service routine share the same variables.

Listing 6.2 — Exercise 4: Software timers and the timer interrupt callback (timer.c)

```
/*
 * timer.c
 *
 * Software timers + the Timer 2 interrupt callback (flowchart FC1).
 */

#include "main.h"
#include "timer.h"
#include "input_reading.h"

static volatile int32_t timerCounter[NO_OF_TIMERS];
static volatile uint8_t timerFlag[NO_OF_TIMERS];

void setTimer(uint8_t index, uint32_t duration_ms)
{
    if (index >= NO_OF_TIMERS) return;

    /* setTimer() runs in the main loop while timer_run() runs in the interrupt:
     * the two writes must not be separated by a tick, so the interrupt is
     * masked for a moment (the previous mask state is restored afterwards). */
    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    timerCounter[index] = (int32_t)ms_to_ticks(duration_ms);
    timerFlag[index] = 0;
    __set_PRIMASK(primask);
}

uint8_t isTimerExpired(uint8_t index)
{
    if (index >= NO_OF_TIMERS) return 0;

    uint8_t expired = 0;
    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    if (timerFlag[index]) {
        timerFlag[index] = 0;
        expired = 1;
    }
    __set_PRIMASK(primask);
    return expired;
}

void timer_run(void)
{
    for (uint8_t i = 0; i < NO_OF_TIMERS; i++) {
        if (timerCounter[i] > 0) {
            timerCounter[i]--;
            if (timerCounter[i] == 0) {
                timerFlag[i] = 1;
            }
```

```
        }
    }
}

/* FC1: Timer 2 interrupt, every TIMER_CYCLE ms.
 * Only light work is done here; all the logic runs in the main loop. */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {
        button_reading();   /* read + debounce the 3 buttons */
        timer_run();        /* update all software timers    */
    }
}
```

To change the interrupt period only `TIMER_CYCLE` and the two CubeMX values have to be changed (Table 6.1); the 2 Hz blinking is generated by the 250 ms software timer and therefore stays at 2 Hz.

Table 6.1 — CubeMX values for another timer interrupt period (HSI 8 MHz)

TIMER_CYCLE	Prescaler	Counter Period
1 ms	799	9
10 ms (used)	7999	9
100 ms	7999	99

Table 6.2 — Number of ticks produced by ms_to_ticks() for each time constant

Constant	Duration	1 ms	10 ms	100 ms
ONE_SECOND_MS	1000 ms	1000	100	10
BLINK_HALF_PERIOD_MS	250 ms	250	25	3 (= 300 ms)
HOLD_TIME_MS	1000 ms	1000	100	10
AUTO_INCREASE_MS	500 ms	500	50	5
SCAN_PERIOD_MS	10 ms	10	1	1 (= 100 ms)

Two limits of the method should be noted. First, a duration can only be a multiple of the timer period: with a 100 ms interrupt the 250 ms half-period is rounded to 3 ticks (300 ms) and the 10 ms digit time becomes 100 ms, so the display would flicker; the periods 1 ms, 5 ms, 10 ms, 25 ms and 50 ms divide all the durations except the scan time exactly. Second, the debouncing window is two samples, i.e. 2 × TIMER_CYCLE, so with a 1 ms interrupt it no longer covers the 10 ms contact bounce of a mechanical button and a larger N would be needed.

7. Exercise 5: Adding Code for Button Debouncing

The example of the handout reads one button; here it is extended to the three buttons of the crossing. `input_reading.c` describes every button by a port/pin pair, reads it every `TIMER_CYCLE` ms in `button_reading()` and keeps two previous samples (`debounceButtonBuffer1` and `debounceButtonBuffer2`). The debounced value

`buttonBuffer` is updated only when the two samples are equal. While a button is pressed a counter measures one second; when it is reached the flag `flagForButtonPress1s` is set. `button_init()` sets all buffers to the released level, because the zero value of a static variable (`GPIO_PIN_RESET`) would otherwise look like a pressed button just after reset.

Listing 7.1 — Exercise 5: Button reading and debouncing (input_reading.c)

```
/*
 * input_reading.c
 *
 * Reads BUTTON_1..BUTTON_3 every TIMER_CYCLE ms with a 2-sample debounce
 * filter (Program 1.2 of the lab, extended to 3 buttons).
 */

#include "main.h"
#include "global.h"
#include "input_reading.h"

/* The timer interrupt period is TIMER_CYCLE ms, so to pass 1 second the
 * interrupt service routine has to run 1000 / TIMER_CYCLE times. */
#define DURATION_FOR_AUTO_INCREASING   ms_to_ticks(HOLD_TIME_MS)
#define BUTTON_IS_PRESSED              GPIO_PIN_RESET
#define BUTTON_IS_RELEASED             GPIO_PIN_SET

/* Hardware description: one port/pin pair for each button. */
static GPIO_TypeDef * const buttonPort[NO_OF_BUTTONS] = {
    BUTTON_1_GPIO_Port, BUTTON_2_GPIO_Port, BUTTON_3_GPIO_Port
};
static const uint16_t buttonPin[NO_OF_BUTTONS] = {
    BUTTON_1_Pin, BUTTON_2_Pin, BUTTON_3_Pin
};

/* The buffer that the final (debounced) result is stored in. */
static volatile GPIO_PinState buttonBuffer[NO_OF_BUTTONS];
/* Two buffers used for debouncing. */
static GPIO_PinState debounceButtonBuffer1[NO_OF_BUTTONS];
static GPIO_PinState debounceButtonBuffer2[NO_OF_BUTTONS];
/* Flag for a button pressed for more than 1 second. */
static volatile uint8_t flagForButtonPress1s[NO_OF_BUTTONS];
/* Counter for the 1 second hold detection. */
static uint16_t counterForButtonPress1s[NO_OF_BUTTONS];

/* All buffers start as "released". Without this the zero-initialised buffers
 * (GPIO_PIN_RESET = pressed) would look like a button press after reset. */
void button_init(void)
{
    for (uint8_t i = 0; i < NO_OF_BUTTONS; i++) {
        buttonBuffer[i]          = BUTTON_IS_RELEASED;
        debounceButtonBuffer1[i] = BUTTON_IS_RELEASED;
        debounceButtonBuffer2[i] = BUTTON_IS_RELEASED;
        flagForButtonPress1s[i]  = 0;
        counterForButtonPress1s[i] = 0;
    }
}

void button_reading(void)
{
    for (uint8_t i = 0; i < NO_OF_BUTTONS; i++) {
        debounceButtonBuffer2[i] = debounceButtonBuffer1[i];
        debounceButtonBuffer1[i] = HAL_GPIO_ReadPin(buttonPort[i], buttonPin[i]);

        if (debounceButtonBuffer1[i] == debounceButtonBuffer2[i]) {
            buttonBuffer[i] = debounceButtonBuffer1[i];

            if (buttonBuffer[i] == BUTTON_IS_PRESSED) {
                /* if a button is pressed, we start counting */
                if (counterForButtonPress1s[i] < DURATION_FOR_AUTO_INCREASING) {
```

```
                    counterForButtonPress1s[i]++;
                } else {
                    /* the flag is turned on when 1 second has passed
                     * since the button is pressed */
                    flagForButtonPress1s[i] = 1;
                }
            } else {
                counterForButtonPress1s[i] = 0;
                flagForButtonPress1s[i] = 0;
            }
        }
    }
}

unsigned char is_button_pressed(unsigned char index)
{
    if (index >= NO_OF_BUTTONS) return 0;
    return (buttonBuffer[index] == BUTTON_IS_PRESSED);
}

unsigned char is_button_pressed_1s(unsigned char index)
{
    if (index >= NO_OF_BUTTONS) return 0;
    return (flagForButtonPress1s[index] == 1);
}
```

`input_processing.c` contains one FSM per button (Figure 3.3). Instead of changing a value directly, the FSM produces an event: `buttonEvent[i]` is set once when the button goes from released to pressed, and for BUTTON_2 again every 500 ms after one second of pressing (software timer `TIMER_AUTO`). The application reads an event with `isButtonEvent()`, which clears it, so each press is processed exactly once. `clearButtonEvent()` throws away an event that is no longer meaningful.

Listing 7.2 — Exercise 5: FSM of the buttons and button events (input_processing.c)

```
/*
 * input_processing.c
 *
 * One 3-state FSM per button (same FSM as Figure 1.9):
 *   State 0 BUTTON_RELEASED                  : button released
 *   State 1 BUTTON_PRESSED                   : just pressed -> fire one event
 *   State 2 BUTTON_PRESSED_MORE_THAN_1_SECOND: held > 1 s; only BUTTON_2 repeats
 *                                              its event every 500 ms
 */

#include "main.h"
#include "input_reading.h"
#include "input_processing.h"
#include "timer.h"

enum ButtonState {
    BUTTON_RELEASED,
    BUTTON_PRESSED,
    BUTTON_PRESSED_MORE_THAN_1_SECOND
};

static enum ButtonState buttonState[NO_OF_BUTTONS];   /* all start in BUTTON_RELEASED */
static volatile uint8_t buttonEvent[NO_OF_BUTTONS];

void fsm_for_input_processing(void)
{
    for (uint8_t i = 0; i < NO_OF_BUTTONS; i++) {
        switch (buttonState[i]) {
```

```
        case BUTTON_RELEASED:
            if (is_button_pressed(i)) {
                buttonState[i] = BUTTON_PRESSED;
                buttonEvent[i] = 1;                    /* the button was just pressed */
            }
            break;

        case BUTTON_PRESSED:
            if (!is_button_pressed(i)) {
                buttonState[i] = BUTTON_RELEASED;
            } else if (is_button_pressed_1s(i)) {
                buttonState[i] = BUTTON_PRESSED_MORE_THAN_1_SECOND;
                if (i == BTN_INC) {
                    setTimer(TIMER_AUTO, AUTO_INCREASE_MS);
                }
            }
            break;

        case BUTTON_PRESSED_MORE_THAN_1_SECOND:
            if (!is_button_pressed(i)) {
                buttonState[i] = BUTTON_RELEASED;
            } else if (i == BTN_INC && isTimerExpired(TIMER_AUTO)) {
                buttonEvent[i] = 1;                    /* count up automatically */
                setTimer(TIMER_AUTO, AUTO_INCREASE_MS);
            }
            break;
        }
    }
}

uint8_t isButtonEvent(uint8_t index)
{
    if (index >= NO_OF_BUTTONS) return 0;
    if (buttonEvent[index]) {
        buttonEvent[index] = 0;
        return 1;
    }
    return 0;
}

void clearButtonEvent(uint8_t index)
{
    if (index >= NO_OF_BUTTONS) return;
    buttonEvent[index] = 0;
}
```

Finally, the first button changes the mode (FC2). The function below is called in the super loop; the mode goes 1 → 2 → 3 → 4 → 1.

Listing 7.3 — Exercise 5: Increasing the mode with BUTTON_1 (traffic_control.c)

```
/* FC2: BUTTON_1 cycles 1 -> 2 -> 3 -> 4 -> 1 */
void traffic_mode_control(void)
{
    if (isButtonEvent(BTN_MODE)) {
        enterMode((mode >= MODE_GREEN) ? MODE_NORMAL : (uint8_t)(mode + 1));
    }
}
```

8. Exercise 6: Adding Code for Displaying Modes

`led_display.c` is the output driver. It hides the hardware details (active-low LEDs, common-anode 7-segment LEDs, PNP digit enables) behind a few functions: `setRoadA()` and

`setRoadB()` light one colour of a road (North + South, or East + West), `allLightsOff()` switches all 12 LEDs off, and `setBlinkColor()` switches all four LEDs of one colour on or off while the other colours stay off.

Listing 8.1 — Exercise 6: Output driver of the traffic LEDs (led_display.c)

```
/*
 * led_display.c
 *
 * Output driver. Signal levels (all active-low):
 *   traffic LED : anode on 3.3 V, cathode on the MCU pin -> lit at 0
 *   7-segment   : common anode                           -> segment lit at 0
 *   digit enable: PNP transistor                         -> digit on at 0
 */

#include "main.h"
#include "led_display.h"
#include "timer.h"

#define LED_ON       GPIO_PIN_RESET
#define LED_OFF      GPIO_PIN_SET
#define SEG_ON       GPIO_PIN_RESET
#define SEG_OFF      GPIO_PIN_SET
#define DIGIT_ON     GPIO_PIN_RESET
#define DIGIT_OFF    GPIO_PIN_SET

/* ------------------------------ traffic LEDs ------------------------------ */

enum { DIR_NORTH = 0, DIR_SOUTH = 1, DIR_EAST = 2, DIR_WEST = 3, NO_OF_DIRECTIONS = 4 };
#define NO_OF_COLORS   3

/* ledPort[colour][direction], colour = LightColor, direction = DIR_xxx */
static GPIO_TypeDef * const ledPort[NO_OF_COLORS][NO_OF_DIRECTIONS] = {
    /* RED */ { RED_NORTH_GPIO_Port, RED_SOUTH_GPIO_Port, RED_EAST_GPIO_Port, RED_WEST_GPIO_Port },
    /* YELLOW */ { YELLOW_NORTH_GPIO_Port, YELLOW_SOUTH_GPIO_Port, YELLOW_EAST_GPIO_Port,
YELLOW_WEST_GPIO_Port },
    /* GREEN */ { GREEN_NORTH_GPIO_Port, GREEN_SOUTH_GPIO_Port, GREEN_EAST_GPIO_Port,
GREEN_WEST_GPIO_Port }
};
static const uint16_t ledPin[NO_OF_COLORS][NO_OF_DIRECTIONS] = {
    /* RED    */ { RED_NORTH_Pin,    RED_SOUTH_Pin,    RED_EAST_Pin,    RED_WEST_Pin    },
    /* YELLOW */ { YELLOW_NORTH_Pin, YELLOW_SOUTH_Pin, YELLOW_EAST_Pin, YELLOW_WEST_Pin },
    /* GREEN  */ { GREEN_NORTH_Pin,  GREEN_SOUTH_Pin,  GREEN_EAST_Pin,  GREEN_WEST_Pin  }
};

static void ledWrite(uint8_t color, uint8_t dir, uint8_t on)
{
    HAL_GPIO_WritePin(ledPort[color][dir], ledPin[color][dir], on ? LED_ON : LED_OFF);
}

/* Light exactly one colour on the two directions that start at firstDir. */
static void roadWrite(uint8_t firstDir, LightColor color)
{
    for (uint8_t c = 0; c < NO_OF_COLORS; c++) {
        ledWrite(c, firstDir,     (c == color));
        ledWrite(c, firstDir + 1, (c == color));
    }
}

void setRoadA(LightColor color) { roadWrite(DIR_NORTH, color); }   /* NORTH + SOUTH */
void setRoadB(LightColor color) { roadWrite(DIR_EAST,  color); }   /* EAST  + WEST  */

void allLightsOff(void)
{
    for (uint8_t c = 0; c < NO_OF_COLORS; c++) {
        for (uint8_t d = 0; d < NO_OF_DIRECTIONS; d++) {
            ledWrite(c, d, 0);
        }
```

```
    }
}

void setBlinkColor(LightColor color, uint8_t on)
{
    for (uint8_t c = 0; c < NO_OF_COLORS; c++) {
        for (uint8_t d = 0; d < NO_OF_DIRECTIONS; d++) {
            ledWrite(c, d, (c == color) ? on : 0);
        }
    }
}
```

The four 7-segment LEDs are multiplexed. The output buffer `led_buffer[4]` holds the four digits (road A tens and units, road B tens and units) and is written with `updateLedBuffer()`. `display_scan()` is called in the super loop; every time the software timer `TIMER_SCAN` expires (10 ms) it switches all digits off, writes the segments of the next digit and enables it, so the four digits are refreshed at 25 Hz each without ghosting.

Listing 8.2 — Exercise 6: 7-segment output driver and scanning (led_display.c)

```
/* ---------------------------- 7-segment LEDs ------------------------------ */

#define NO_OF_DIGITS   4
#define NO_OF_SEGMENTS 7

static GPIO_TypeDef * const segPort[NO_OF_SEGMENTS] = {
    SEG_0_GPIO_Port, SEG_1_GPIO_Port, SEG_2_GPIO_Port, SEG_3_GPIO_Port,
    SEG_4_GPIO_Port, SEG_5_GPIO_Port, SEG_6_GPIO_Port
};
static const uint16_t segPin[NO_OF_SEGMENTS] = {
    SEG_0_Pin, SEG_1_Pin, SEG_2_Pin, SEG_3_Pin, SEG_4_Pin, SEG_5_Pin, SEG_6_Pin
};

/* digit index: 0 = road A tens, 1 = road A units, 2 = road B tens, 3 = road B units */
static GPIO_TypeDef * const enPort[NO_OF_DIGITS] = {
    EN_NS_TENS_GPIO_Port, EN_NS_UNITS_GPIO_Port, EN_EW_TENS_GPIO_Port, EN_EW_UNITS_GPIO_Port
};
static const uint16_t enPin[NO_OF_DIGITS] = {
    EN_NS_TENS_Pin, EN_NS_UNITS_Pin, EN_EW_TENS_Pin, EN_EW_UNITS_Pin
};

/* Common-anode code: bit i corresponds to SEG_i (a..g), a bit equal to 0 lights the segment. */
static const uint8_t segCode[10] = {
    0x40, /* 0 */  0x79, /* 1 */  0x24, /* 2 */  0x30, /* 3 */  0x19, /* 4 */
    0x12, /* 5 */  0x02, /* 6 */  0x78, /* 7 */  0x00, /* 8 */  0x10  /* 9 */
};

static uint8_t led_buffer[NO_OF_DIGITS];     /* the output buffer of the 7-segment LEDs */
static uint8_t scanIndex = 0;

static void display7SEG(uint8_t number)
{
    uint8_t code = (number <= 9) ? segCode[number] : 0x7F;   /* 0x7F = blank */
    for (uint8_t s = 0; s < NO_OF_SEGMENTS; s++) {
        HAL_GPIO_WritePin(segPort[s], segPin[s], ((code >> s) & 1) ? SEG_OFF : SEG_ON);
    }
}

void updateLedBuffer(int valueA, int valueB)
{
    valueA %= 100;
    valueB %= 100;
    led_buffer[0] = (uint8_t)(valueA / 10);
    led_buffer[1] = (uint8_t)(valueA % 10);
    led_buffer[2] = (uint8_t)(valueB / 10);
```

```
    led_buffer[3] = (uint8_t)(valueB % 10);
}

void display_init(void)
{
    allLightsOff();
    for (uint8_t s = 0; s < NO_OF_SEGMENTS; s++) {
        HAL_GPIO_WritePin(segPort[s], segPin[s], SEG_OFF);
    }
    for (uint8_t d = 0; d < NO_OF_DIGITS; d++) {
        HAL_GPIO_WritePin(enPort[d], enPin[d], DIGIT_OFF);
    }
    updateLedBuffer(0, 0);
    scanIndex = 0;
    setTimer(TIMER_SCAN, SCAN_PERIOD_MS);
}

/* Show the next digit each time the scan timer expires (4 digits, one at a time). */
void display_scan(void)
{
    if (!isTimerExpired(TIMER_SCAN)) return;
    setTimer(TIMER_SCAN, SCAN_PERIOD_MS);

    for (uint8_t d = 0; d < NO_OF_DIGITS; d++) {              /* turn all digits off first, */
        HAL_GPIO_WritePin(enPort[d], enPin[d], DIGIT_OFF);    /* so there is no ghosting    */
    }
    display7SEG(led_buffer[scanIndex]);
    HAL_GPIO_WritePin(enPort[scanIndex], enPin[scanIndex], DIGIT_ON);

    scanIndex = (scanIndex + 1) % NO_OF_DIGITS;
}
```

The display of the modes is done in `enterMode()`, which is called each time the mode changes. In mode 1 it restarts the traffic light cycle. In modes 2–4 it copies the current duration into the draft value `draft`, switches all LEDs off, starts the 2 Hz blinking of the colour returned by `modeColor()` (red, yellow or green) and writes the draft value on the left pair of digits and the mode number on the right pair of digits. The blinking itself is done in `traffic_edit()` (Listing 9.1) with the software timer `TIMER_BLINK`, which toggles the LEDs every 250 ms, that is 2 Hz.

Listing 8.3 — Exercise 6: Selecting the display and the blinking colour of a mode (traffic_control.c)

```
/* ---- modes 2..4 : value being edited and blink state ---- */
static int     draft;
static uint8_t blinkOn;

static int appliedValue(uint8_t m)
{
    if (m == MODE_RED)    return redTime;
    if (m == MODE_YELLOW) return yellowTime;
    return greenTime;
}

static LightColor modeColor(uint8_t m)
{
    if (m == MODE_RED)    return LIGHT_RED;
    if (m == MODE_YELLOW) return LIGHT_YELLOW;
    return LIGHT_GREEN;
}

static void enterMode(uint8_t newMode)
{
    mode = newMode;
    /* events produced before this moment belong to the previous mode */
```

```
    clearButtonEvent(BTN_INC);
    clearButtonEvent(BTN_SET);

    if (mode == MODE_NORMAL) {
        startCycle();
    } else {
        draft = appliedValue(mode);  /* a draft left over from before is discarded */
        blinkOn = 1;
        allLightsOff();
        setBlinkColor(modeColor(mode), blinkOn);
        updateLedBuffer(draft, mode);
        setTimer(TIMER_BLINK, BLINK_HALF_PERIOD_MS);
    }
}
```

9. Exercise 7: Increasing the RED Duration

In mode 2 the second button increases the draft value of the red duration and the third button sets it (FC4). The draft is increased in the range 1–99; after 99 it returns to 1. BUTTON_3 stores the draft in `redTime` only if the new set of durations is valid ($R > Y$ and $G + Y \leq 99$, see Exercise 1) and then returns to mode 1, which restarts the cycle with the new value. If the value is not valid, the system stays in mode 2 so that the user can correct it. Holding BUTTON_2 for more than one second increases the draft every 500 ms (Exercise 5).

Listing 9.1 — Exercise 7: Editing a duration, FC4 (traffic_control.c)

```
uint8_t mode = MODE_NORMAL;

/* ---- durations in use (seconds) ---- */
static int redTime    = DEFAULT_RED_TIME;
static int yellowTime = DEFAULT_YELLOW_TIME;
static int greenTime  = DEFAULT_GREEN_TIME;

/* The new set of durations must keep both roads consistent. */
static uint8_t isValid(int r, int y, int g)
{
    return (r > y) && (g + y <= MAX_TIME_VALUE);
}

/* FC4: modes 2, 3, 4 */
void traffic_edit(void)
{
    if (isTimerExpired(TIMER_BLINK)) {
        setTimer(TIMER_BLINK, BLINK_HALF_PERIOD_MS);
        blinkOn = !blinkOn;
        setBlinkColor(modeColor(mode), blinkOn);
    }

    if (isButtonEvent(BTN_INC)) {
        draft = (draft >= MAX_TIME_VALUE) ? MIN_TIME_VALUE : draft + 1;
        updateLedBuffer(draft, mode);
    }

    if (isButtonEvent(BTN_SET)) {
        int r = redTime, y = yellowTime, g = greenTime;
        if (mode == MODE_RED)         r = draft;
        else if (mode == MODE_YELLOW) y = draft;
        else                          g = draft;

        if (isValid(r, y, g)) {
            redTime = r; yellowTime = y; greenTime = g;
            enterMode(MODE_NORMAL);       /* restart the cycle with the new values */
```

```
        }
        /* invalid: stay in this mode, the user can correct the value */
    }
}
```

Table 9.1 — Example: setting the RED duration from 5 s to 8 s

Action	Left digits	Right digits	LEDs
BUTTON_1 (mode 1 → 2)	05	02	All red LEDs blink at 2 Hz
BUTTON_2 pressed 3 times	08	02	Red LEDs blink
BUTTON_2 held for more than 1 s	+1 every 500 ms	02	Red LEDs blink
BUTTON_3 (draft = 8, valid: 8 > Y = 2)	08	06	Mode 1: road A red (8 s), road B green (6 s)

10. Exercise 8: Increasing the YELLOW (Amber) Duration

Mode 3 uses the same function FC4 with `mode = MODE_YELLOW`: `modeColor()` makes the yellow LEDs blink, `appliedValue()` loads `yellowTime` as the initial draft value and, when BUTTON_3 is pressed, the draft is assigned to the yellow duration only (Listing 10.1). Since the red time of road A must stay longer than the yellow time, the value is refused when the draft is not smaller than `redTime`; with R = 5 the largest accepted yellow time is 4 s.

Listing 10.1 — Exercise 8: Part of FC4 that sets the YELLOW duration (traffic_control.c)

```
    if (isButtonEvent(BTN_SET)) {
        int r = redTime, y = yellowTime, g = greenTime;
        if (mode == MODE_RED)         r = draft;
        else if (mode == MODE_YELLOW) y = draft;
        else                          g = draft;

        if (isValid(r, y, g)) {
            redTime = r; yellowTime = y; greenTime = g;
            enterMode(MODE_NORMAL);       /* restart the cycle with the new values */
        }
        /* invalid: stay in this mode, the user can correct the value */
    }
```

Table 10.1 — Examples of setting the YELLOW duration (R = 5, G = 3)

Draft Y	Test	Result
3	5 > 3 and 3 + 3 ≤ 99	Accepted. Road B: green 2 s, yellow 3 s, red 6 s; cycle 5 + 3 + 3 = 11 s
5	5 > 5 is false	Refused, the system stays in mode 3

11. Exercise 9: Increasing the GREEN Duration

Mode 4 is the last case of FC4: the green LEDs blink, the initial draft is `greenTime` and BUTTON_3 assigns the draft to the green duration. The condition G + Y ≤ 99 keeps the red time of road B (G + Y) on two digits.

Listing 11.1 — Exercise 9: Validation of the new durations (traffic_control.c)

```
/* The new set of durations must keep both roads consistent. */
static uint8_t isValid(int r, int y, int g)
{
    return (r > y) && (g + y <= MAX_TIME_VALUE);
}
```

Table 11.1 — Examples of setting the GREEN duration (R = 5, Y = 2)

Draft G	Test	Result
5	5 + 2 ≤ 99	Accepted. Road B: green 3 s, yellow 2 s, red 7 s; cycle 5 + 5 + 2 = 12 s
98	98 + 2 = 100 > 99	Refused, the system stays in mode 4

12. Exercise 10: Finishing the Project

All the previous parts are integrated in the super loop of `main.c`. The peripherals are initialised first, then `display_init()`, `button_init()` and `traffic_init()` put the system in a known state (all LEDs off, all buttons released, R = 5, Y = 2, G = 3, mode 1) and finally the timer interrupt is started. Inside the loop the buttons are converted into events, the mode is updated, the function of the current mode is executed and the 7-segment LEDs are refreshed. The interrupt only reads the buttons and updates the software timers (FC1); all the logic runs in the main loop.

Listing 12.1 — Exercise 10: Initialisation and super loop (main.c)

```
/* USER CODE BEGIN Includes */
#include "global.h"
#include "timer.h"
#include "input_reading.h"
#include "input_processing.h"
#include "led_display.h"
#include "traffic_control.h"
/* USER CODE END Includes */


  /* USER CODE BEGIN 2 */
  display_init();                   /* all LEDs off, scan timer ready          */
  button_init();                    /* all buttons "released"                  */
  traffic_init();                   /* R = 5, Y = 2, G = 3, mode 1, start cycle */
  HAL_TIM_Base_Start_IT(&htim2);    /* Timer 2 interrupt every 10 ms           */
  /* USER CODE END 2 */


  while (1)
  {
    /* USER CODE END WHILE */


    /* USER CODE BEGIN 3 */
    fsm_for_input_processing();     /* buttons -> events                       */
    traffic_mode_control();         /* BUTTON_1 changes the mode               */
```

```
    if (mode == MODE_NORMAL) {
      traffic_normal();             /* FC3                                    */
    } else {
      traffic_edit();               /* FC4                                    */
    }
    display_scan();                 /* refresh the 4 seven-segment LEDs       */
  }
  /* USER CODE END 3 */
```

In mode 1, `traffic_normal()` (FC3) is called. It ignores BUTTON_2 and BUTTON_3 (their events are cleared so that they cannot wait for a later mode) and, every second, decrements `remaining`. When it reaches 0 the phase changes (red → green → yellow → red) and `showNormal()` writes the LEDs and the two counters of the 7-segment LEDs from `phase` and `remaining`, with the road B values defined in Exercise 1.

Listing 12.2 — Exercise 10: Traffic light of mode 1, FC3 (traffic_control.c)

```
/* ---- mode 1 : state of the traffic lights ---- */
typedef enum { PHASE_A_RED, PHASE_A_GREEN, PHASE_A_YELLOW } Phase;
static Phase phase;
static int   remaining;          /* seconds left in the current phase of road A */

/* Output buffer + LEDs of mode 1, derived from phase and remaining. */
static void showNormal(void)
{
    int valueB;
    switch (phase) {
    case PHASE_A_RED:
        setRoadA(LIGHT_RED);
        if (remaining > yellowTime) { setRoadB(LIGHT_GREEN); valueB = remaining - yellowTime; }
        else                        { setRoadB(LIGHT_YELLOW); valueB = remaining; }
        break;
    case PHASE_A_GREEN:
        setRoadA(LIGHT_GREEN);
        setRoadB(LIGHT_RED);
        valueB = remaining + yellowTime;
        break;
    default: /* PHASE_A_YELLOW */
        setRoadA(LIGHT_YELLOW);
        setRoadB(LIGHT_RED);
        valueB = remaining;
        break;
    }
    updateLedBuffer(remaining, valueB);
}

static void startCycle(void)
{
    phase = PHASE_A_RED;             /* A red, B green */
    remaining = redTime;
    showNormal();
    setTimer(TIMER_SECOND, ONE_SECOND_MS);
}

void traffic_init(void)
{
    redTime    = DEFAULT_RED_TIME;
    yellowTime = DEFAULT_YELLOW_TIME;
    greenTime  = DEFAULT_GREEN_TIME;
    enterMode(MODE_NORMAL);
}

/* FC3: mode 1 */
void traffic_normal(void)
{
    /* BUTTON_2 / BUTTON_3 mean nothing here: do not let an event wait for later */
    clearButtonEvent(BTN_INC);
```

```
    clearButtonEvent(BTN_SET);

    if (!isTimerExpired(TIMER_SECOND)) return;
    setTimer(TIMER_SECOND, ONE_SECOND_MS);

    if (--remaining <= 0) {
        switch (phase) {
        case PHASE_A_RED:    phase = PHASE_A_GREEN;  remaining = greenTime;  break;
        case PHASE_A_GREEN:  phase = PHASE_A_YELLOW; remaining = yellowTime; break;
        default:             phase = PHASE_A_RED;    remaining = redTime;    break;
        }
    }
    showNormal();
}
```

The remaining constants of the application are in `global.h` (Listing 12.3), and the four software timers used by the program are summarised in Table 12.1.

Listing 12.3 — Exercise 10: Modes, default durations and buttons (global.h)

```
/* ---- application modes ------------------------------------------------------ */
#define MODE_NORMAL            1
#define MODE_RED               2
#define MODE_YELLOW            3
#define MODE_GREEN             4

/* ---- default durations (seconds) and allowed range -------------------------- */
#define DEFAULT_RED_TIME       5
#define DEFAULT_YELLOW_TIME    2
#define DEFAULT_GREEN_TIME     3
#define MIN_TIME_VALUE         1
#define MAX_TIME_VALUE         99

/* ---- buttons ---------------------------------------------------------------- */
#define NO_OF_BUTTONS          3
#define BTN_MODE               0      /* BUTTON_1 */
#define BTN_INC                1      /* BUTTON_2 */
#define BTN_SET                2      /* BUTTON_3 */
```

Table 12.1 — Software timers

Timer	Period	Used by
TIMER_SECOND	1000 ms	Traffic light tick in mode 1 (traffic_normal)
TIMER_BLINK	250 ms	Toggle of the blinking LEDs in modes 2–4 (2 Hz)
TIMER_SCAN	10 ms	Time of one 7-segment digit (display_scan)
TIMER_AUTO	500 ms	Auto-repeat of BUTTON_2 after 1 s (fsm_for_input_processing)

Table 12.2 lists the scenarios used to check all the features of the specification.

Table 12.2 — Test scenarios and expected behaviour

No.	Action	Expected behaviour
1	Power-up, no button pressed	Mode 1 with R = 5, Y = 2, G = 3 and the cycle of Table 3.2
2	BUTTON_1 once	Mode 2: all red LEDs blink at 2 Hz, left digits show 05, right digits show 02
3	BUTTON_2 once, then held more than 1 s	Value +1 on the press, then +1 every 500 ms; 99 is followed by 01
4	BUTTON_3 with a valid value	The value is stored and the system returns to mode 1 with the new cycle
5	BUTTON_1 again in mode 2, then again in mode 3	Modes 3 (yellow, right digits 03) and 4 (green, right digits 04) are reached in order
6	BUTTON_3 with an invalid value (R ≤ Y or G + Y > 99)	The value is refused, the system stays in the same edit mode
7	BUTTON_1 in mode 4	Return to mode 1; the draft value is discarded and the previous durations are kept
8	BUTTON_2 or BUTTON_3 in mode 1	No effect

Demo video:
https://drive.google.com/file/d/1oFzTM_DSl30pyX2pBSSOp0p_vEkG6Taf/view?usp=sharing

Lab report: